

Học Sâu (Deep Learning)

Chương 7: Lập trình Chuyên sâu với Keras (A Deep Dive on Keras)

Đại học Đông Á

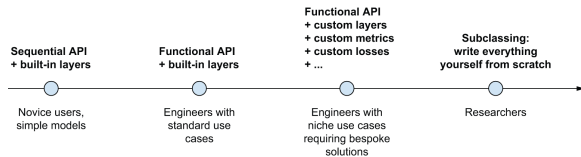
July 23, 2026

Nội dung chương

1. Triết lý Thiết kế của Keras
2. Ba phương pháp Xây dựng Mô hình
3. Tùy biến Vòng lặp Huấn luyện (Customizing)
4. Callbacks: Tự động hóa quá trình Huấn luyện
5. Giám sát với TensorBoard
6. Tổng kết

Từng bước phơi bày sự phức tạp (Progressive Disclosure)

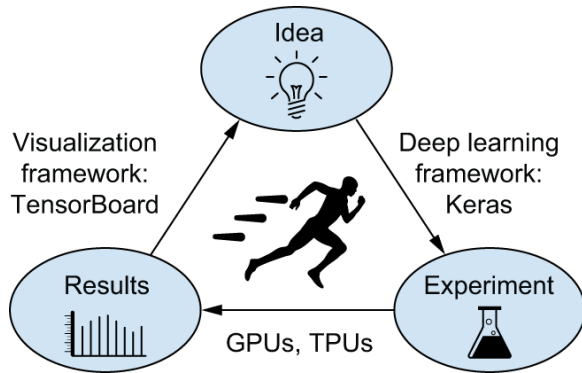
- **Nguyên tắc thiết kế Keras:** Làm cho mọi thứ trở nên dễ bắt đầu, nhưng vẫn hoàn toàn khả thi để xử lý các bài toán siêu phức tạp.
- Không có một "con đường duy nhất". Keras cung cấp một **Phổ quy trình (Spectrum of workflows)**.
- Người mới dùng Keras như Scikit-learn ('fit()'). Chuyên gia dùng Keras như NumPy (viết tay mọi thứ từ đầu).



Hình 7.1: Từ Sequential đến Model Subclassing

Vòng lặp Tiến bộ (The Loop of Progress)

- Khoa học là quá trình thử nghiệm lặp đi lặp lại.
- **Idea:** Khởi tạo ý tưởng.
- **Experiment:** Cài đặt mã nguồn.
- **Evaluate:** Đánh giá kết quả → Rút ra bài học và tạo Idea mới.
- Keras được tối ưu để thu ngắn tối đa thời gian từ lúc có ý tưởng đến lúc chạy xong thí nghiệm.



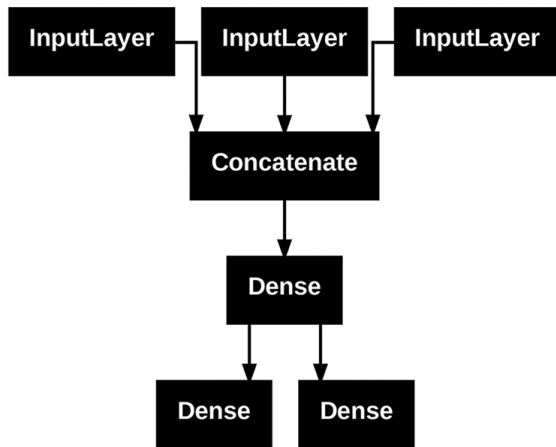
Hình 7.2: Vòng lặp phát triển Mô hình ML

Phương pháp 1: Sequential API

- Cấu trúc dữ liệu dạng **Danh sách (Python List)**.
- Đơn giản nhất, dễ tiếp cận nhất.
- **Hạn chế:** Chỉ hỗ trợ các lớp xếp chồng tuyến tính (lớp sau nối tiếp lớp trước).
- Không thể áp dụng cho: Mô hình đa đầu vào (Multi-input), đa đầu ra (Multi-output), hay các cấu trúc phân nhánh (vd: ResNet).

Phương pháp 2: Functional API (Mạng phân loại vé)

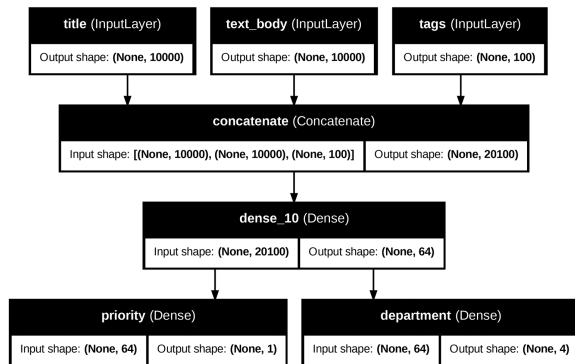
- Coi các layer là các **Hàm (Functions)**.
Nhận Tensor và trả về Tensor.
- Cho phép xây dựng mạng **Đồ thị đa hướng (Directed Acyclic Graph)**.
- Ví dụ: Hệ thống phân loại vé hỗ trợ (Ticket Classifier). Đầu vào: (1) Tiêu đề, (2) Nội dung văn bản, (3) Thẻ tags. Đầu ra: (1) Mức độ ưu tiên, (2) Phòng ban xử lý.



Hình 7.3: Sơ đồ mạng đa luồng

Functional API: Luồng dữ liệu và Shapes

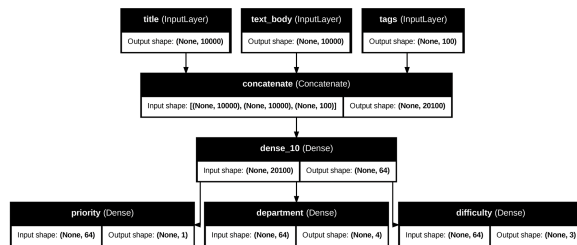
- Lợi thế tối thượng của Functional API là hệ thống **truyền kích thước (Shape Inference)**.
- Ký hiệu TensorShape: '(None, 64)'. Kích thước lô (Batch Size) là 'None' để hệ thống tự thích ứng, '64' là số lượng đặc trưng.
- Bất kỳ sự lệch pha kích thước nào cũng được phát hiện ngay lúc thiết kế (Design Time) thay vì lúc chạy (Runtime).



Hình 7.4: Đồ thị truyền tải Shapes

Functional API: Khả năng Tái sử dụng

- Mô hình Keras cũng là một "Layer khổng lồ".
- Ta có thể trích xuất các *nút trung gian* (*Intermediate Nodes*) để tạo ra một nhánh đồ thị hoàn toàn mới.
- Ví dụ: Thêm bộ phân loại "Mức độ khó" vào sơ đồ phân loại vé có sẵn mà không cần viết lại toàn bộ mã.



Hình 7.5: Mở rộng đồ thị Functional

Phương pháp 3: Model Subclassing

- Kế thừa Lớp trình Hướng đối tượng (OOP) từ lớp cơ sở `keras.Model`.
- Khai báo các trọng số tại phương thức khởi tạo `__init__()`.
- Định nghĩa dòng chảy Toán học ngầm tại phương thức `call()`.
- **Ưu điểm:** Cung cấp khả năng kiểm soát tuyệt đối (Vd: Dùng vòng lặp `for` động bên trong mạng Nơ-ron).
- **Nhược điểm:** Mất đi tính minh bạch của đồ thị. Không thể in ra Tóm tắt cấu trúc (Summary) dễ dàng.

Tự định nghĩa Hàm Mất mát (Custom Losses)

Đôi khi hàm *Mean Squared Error (MSE)* mặc định là không đủ. Ta phải kế thừa lớp `keras.losses.Loss` và cài đặt công thức Toán học:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (1)$$

Trong đó:

- y_i là Mục tiêu gốc (True Label).
- $\hat{y}_i$ là Dự đoán (Prediction).
- Cài đặt qua phương thức `call(y_true, y_pred)`.

Tự định nghĩa Thước đo (Custom Metrics)

Metrics khác với Loss ở chỗ nó mang **Trạng thái (Stateful)**. Ta kế thừa `keras.metrics.Metric`.

- **Update State:** Cộng dồn trạng thái sau mỗi Lô (Batch).

$$S_{total} = S_{total} + \sum(\text{Correct Predictions}) \quad (2)$$

- **Result:** Trả về kết quả đầu ra hiển thị cho người dùng.
- **Reset State:** Reset biến đếm về 0 khi bắt đầu một Epoch mới.

Viết lại Vòng lặp Huấn luyện bằng GradientTape

Thay vì gọi `model.fit()`, ta có thể sử dụng `tf.GradientTape` để tự điều khiển phép tính **Đạo hàm riêng (Backpropagation)**:

$$\Delta W = -\eta \cdot \frac{\partial L}{\partial W} \quad (3)$$

Quy trình lặp:

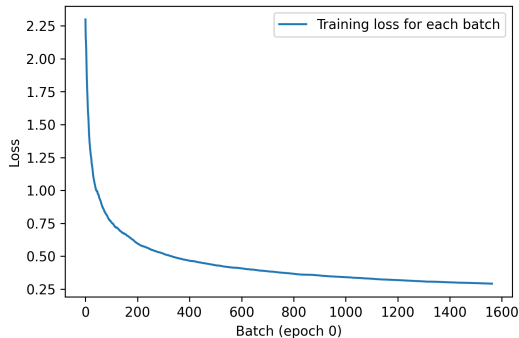
- 1 Mở Tape, truyền dữ liệu qua Mạng.
- 2 Tính Loss L .
- 3 Dùng Tape truy xuất Gradient $\frac{\partial L}{\partial W}$.
- 4 Gửi Gradient cho Trình tối ưu hóa (Optimizer) để cập nhật Trọng số W .

Sức mạnh của Callbacks

- Chạy mô hình hàng chục ngàn Epochs tốn hàng giờ đồng hồ, Kỹ sư không thể ngồi canh bằng mắt.
- **Callbacks** là các "hook" được kích hoạt tự động vào các thời điểm (Đầu/cuối Epoch, Đầu/cuối Batch).
- **EarlyStopping**: Tự động dừng huấn luyện khi Validation Loss có dấu hiệu chạm đáy. (Ngăn Overfitting).
- **ModelCheckpoint**: Tự động sao lưu Trọng số Tốt nhất xuống ổ cứng.
- **ReduceLROnPlateau**: Tự động bóp nhỏ Tốc độ học (Learning Rate) khi Loss bị mắc kẹt.

Tùy biến Callback (Custom Callback)

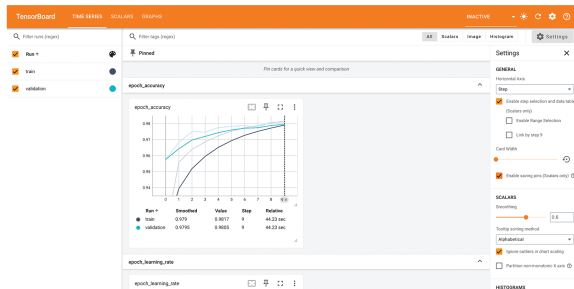
- Bằng cách kế thừa `keras.callbacks.Callback`, bạn có thể can thiệp vào hành vi của hệ thống tại sự kiện `on_epoch_end()`.
- *Ví dụ:* Viết một Callback lưu lại giá trị Loss của từng Batch và vẽ thành Đồ thị phân tán.
- Mật độ dày đặc của biểu đồ cho thấy sự thay đổi Loss ở cấp độ Lô (Batch-level).



Hình 7.6: Đồ thị Loss do Custom Callback sinh ra

Bảng điều khiển TensorBoard

- Mất thường không thể đọc hàng triệu con số trên Terminal.
- **TensorBoard** chạy một Web Server nội bộ, theo dõi và trực quan hóa thời gian thực (Real-time).
- Chức năng:
 - So sánh Metrics (Loss, Accuracy) của hàng chục thí nghiệm khác nhau.
 - Vẽ biểu đồ phân phối trọng số (Histograms) để kiểm tra Gradient có bị Triệt tiêu (Vanishing) hay Bùng nổ (Exploding) hay không.
 - Vẽ lại Đồ thị Mô hình đa luồng phức tạp.



Hình 7.7: Giao diện Giám sát TensorBoard

Keras không chỉ là thư viện dành cho người mới, nó là một Vũ khí Chuyên sâu:

- Chọn đúng công cụ: **Sequential** (Đơn giản), **Functional API** (Đa luồng đồ thị - Khuyến dùng), **Model Subclassing** (Tự điều khiển luồng ngầm).
- Khả năng tùy biến vô hạn: Có thể tự viết lại **Custom Loss**, **Custom Metric** và thậm chí phá vỡ hàm `fit()` để viết **Custom Training Loop**.
- Tự động hóa: Giải phóng Kỹ sư khỏi việc canh màn hình nhờ hệ thống **Callbacks** (EarlyStopping, Checkpoint).
- Giám sát học thuật: Trực quan hóa mọi thứ với **TensorBoard**.